



CS310 – Computer Networks

Semester I, 2026

Mode: Face to Face (Online Submission Required)

Assignment 1 Category: Group Assignment [To be attempted in pairs]

Due Date: April 14th, 2026

Weight: 15%

Assignments Objectives

This assignment tests the following course learning outcomes and the associated CBOK.

Attributes **CLO 4: Develop the network application using socket programming.** To assess the student's Learning Outcome 4, namely, the ability to develop network application using socket programming detailed as:

1. A simple socket server and client
2. A server-client application that functions like a full-fledged socket application, complete with its own custom header and content.
3. Build-in Python or any other programming language is accepted.
4. Prepare presentation and present.

***Note:** The attached marking rubric on pages 7-9 will be used to assess this assignment.

Problem Description

This assignment builds directly on the TCP and UDP socket concepts covered in the labs. You are required to implement a simple **file transfer server and client** that performs reliable data transfer over the network. The server will read an entire file and send all its bytes to the client; the client will receive those bytes and save them as a new file on its local machine.

You must create **two separate Python programs**:

- `fileserver.py` - the server that listens for connections, validates file requests, and sends the file if it exists.
- `client.py` - the client that connects to the server, requests a specific file, receives the data, and saves it locally.

Workflow:

1. The server starts first and waits for a client to connect. Once a client connects, the server must clearly indicate (on the console) that it is **ready for communication**.
2. The client then sends the name of the file it wants to download.
3. On receiving the request, the server checks whether the requested file exists in its working directory:
 - If the file exists, the server sends a positive acknowledgement/permission message to the client and begins transmitting the file contents.
 - If the file does **not** exist, the server sends an error message ("File does not exist"), and the client should display this message.
4. While the file is being transferred, the client must display a **download progress percentage** (e.g., "Downloading... 45%") at regular intervals. You may implement this using a simple progress bar or printed percentage updates (you can decide the exact method; feel free to discuss implementation details during the lab).
5. Once the entire file has been received and saved successfully, both the server and client should print a clear success message ("Download complete" / "File transfer successful").

6. For demonstration and marking purposes, both programs will run on the **same PC** (different terminals). The downloaded file must appear in the client's working directory with the same name as the original (or with a "_downloaded" suffix if you prefer).

Requirements:

1. Use **Python 3** (the same stable version used in the labs).
2. The server must use a **reliable** transport mechanism (TCP is recommended for simplicity, but you may implement reliable transfer over UDP using acknowledgements and retransmissions if you prefer to demonstrate UDP concepts).
3. Hard-code the server IP address as 127.0.0.1 and port number as 5000 (or any free port above 1024). The client must connect to the same address and port.
4. The server should handle only **one client connection at a time** (no multi-threading required).
5. Include appropriate error handling (e.g., connection errors, file I/O errors) and print meaningful messages to the console.
6. The programs must be clearly commented so the marker can understand the flow of socket creation, connection, file request, data transfer, and progress reporting.
7. Test the application by placing a sample file (e.g., test.txt or image.jpg) in the server's folder and verifying that the client can successfully download and open it.

Remember:

This assignment tests your understanding of socket programming, reliable data transfer, client-server interaction, and basic progress monitoring all core skills from the networking labs.

Submission Instruction

- Write a README file for detailed notes regarding the functionality of the corresponding code, and a set of instructions on how to run it.
- Include your name in each code file and also in the README file.
- The code must be commented properly using Python.
- Please test your programs carefully to make sure they work properly before submission.

- Demonstration of the solution is also required for evaluation as a presentation during the assessment.
- Submit your assignment (Text and code) on Moodle on or before the due date.
- This assignment can be submitted in pairs. Assign a leader and submit the assignment through the leader's Moodle account.
- You must submit the two Python files (fileserver.py and client.py) into a single folder named YourName_StudentID_FileTransfer_Assignment. Submit the zipped folder.

LATE SUBMISSION = ZERO (0)

Plagiarism

- Incorrect submissions will result in loss of marks or simply a grade of zero being awarded.
- Late Submission = Grade of Zero (0) for Assignment 1.
- You are urged not to use any AI-generated software to complete this assignment 1.

Zero for any penalty cases, thus no two assignment submissions should produce the same results. Any such case will be dealt with severely, with actions including awarding a mark of zero, forwarding the case to the Head of School, and reporting the matter to the Student Disciplinary Committee.

Queries

- Maintain contact through the Moodle discussion forum named Assignment 1 Discussion Forum
- Feel free to make an appointment via email with the Course Coordinator for any urgent matters regarding Assignment 1.

AI Usage: Zero

Assignment 1 Assessment Rubric

*Ensure to Fill This: Mandatory! Student Contribution Award

This award recognizes students who have significantly made a difference in their actions, whether it is within their specific cohort, in CS310. Failure to submit will result in a group penalty.

Please complete and return the following form together with AI Report and Coding File:

Student Name	Contribution	Justification	Signature

CBOK	Criteria		
A. Programming	Unsatisfactory	Satisfactory	Good
	• Plagiarism • Poor indentation, hard to read and follow the code. • Lots Of Bugs /Errors • Program Produces Unexpected Output • No Proper Encapsulation, • Inheritance Or ☐ Polymorphism • For Object-Oriented • Programming ☐ Hard Coding • Poor Structure	• Able to write complex code for a loosely defined problem • Appropriate structure of the program • Use of standard programming practices such as modularity, low coupling, high cohesion, etc. • Appropriate encapsulation and inheritance or OOP. • The computer program produces the expected output	☐ All satisfactory and demonstrates very good programming skills.
B. Networking			
	☐ Unable to demonstrate a basic understanding of the data communication and networking concepts in the computer system	☐ Able to describe most of the fundamental concepts of networking and data communication	☐ Demonstrates the applicability of the key data communication and networking concepts to open-ended problems in the discipline

The End..